

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №8
по дисциплине «Вычислительная математика»
Тема: Решение систем линейных уравнений методом Якоби.

Студент гр. 4342

Садыков С.Р.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Разработать и исследовать программу решения системы линейных уравнений методом Якоби (методом простых итераций) с выбором ведущего элемента, проверкой существования решения и невырожденности матрицы, а также провести тестирование на базовых, больших и специальных матрицах (в том числе матрицах Гильберта) с анализом корректности, времени и памяти.

Теоретический минимум.

Метод Якоби (метод простых итераций для СЛУ) применяется для решения системы линейных уравнений вида $A * x = b$, где A — матрица коэффициентов размером $n \times n$, x — вектор неизвестных, b — вектор правых частей. Идея метода состоит в том, чтобы не решать систему напрямую, а строить последовательность приближений $x^{(0)}$, $x^{(1)}$, $x^{(2)}$, ..., которая при выполнении условий сходимости стремится к точному решению.

Матрицу представляют в виде $A = D + L + U$, где D — диагональная часть, L — нижняя треугольная часть без диагонали, U — верхняя треугольная часть без диагонали. Тогда итерационная формула Якоби записывается как $x^{(k+1)} = D^{-1} * (b - (L + U) * x^{(k)})$. Покомпонентно: $x_i^{(k+1)} = (b_i - \sum_{j \neq i} (a_{ij} * x_j^{(k)})) / a_{ii}$. Ключевая особенность: на шаге $k+1$ используются только значения предыдущей итерации k .

Для корректной работы метода нужны ненулевые диагональные элементы a_{ii} , иначе формула деления не определена. Поэтому на практике применяют перестановки строк (выбор ведущего элемента), чтобы поставить на диагональ более крупные по модулю элементы и избежать нулей/почти нулей. Это не классический “прямой ход”, как в Гауссе, а предварительное преобразование системы для устойчивых итераций.

Сходимость метода определяется итерационной матрицей $B = -D^{-1}(L+U)$. Метод сходится тогда и только тогда, когда спектральный радиус $\rho(B) < 1$. На практике чаще используют достаточные условия: строгая диагональная преобладанность по строкам $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ гарантирует сходимость Якоби. Если это условие нарушено, метод может сходиться медленно или расходиться.

Проверка существования и числа решений системы выполняется через ранги. Если $\text{rank}(A) = \text{rank}([A|b])$, система совместна. Если при этом $\text{rank}(A) = \text{rank}([A|b]) = n$, решение единственное. Если $\text{rank}(A) = \text{rank}([A|b]) < n$, решений бесконечно много. Если $\text{rank}(A) \neq \text{rank}([A|b])$, система несовместна и решений нет. В вычислениях это проявляется, например, строкой вида $[0 \ 0 \ \dots \ 0 \ | \ c]$, где $c \neq 0$.

Критерий остановки итераций обычно задают по разности соседних приближений $\|x^{(k+1)} - x^{(k)}\|$ или по невязке $r^{(k)} = A * x^{(k)} - b$ и норме $\|r^{(k)}\|$. В численных расчётах используется порог eps : значения с модулем меньше eps считаются нулевыми. Для контроля качества решения оценивают невязку $\|A * x - b\|$, а если известно точное решение x^* , дополнительно ошибку $\|x - x^*\|$.

По трудоёмкости одна итерация метода Якоби для плотной матрицы требует порядка $O(n^2)$ операций, а для разреженной — $O(\text{nnz})$, где nnz — число ненулевых элементов. Общая стоимость составляет $O(\text{iterations} * n^2)$ для плотного случая (или $O(\text{iterations} * \text{nnz})$ для разреженного), поэтому метод особенно удобен для больших разреженных систем.

Выполнение работы.

Программный комплекс имеет модульную структуру и состоит из логически разделенных компонентов, каждый из которых отвечает за определенный этап обработки СЛУ: от предварительного анализа и проверки входных данных до итерационного решения и верификации результата.

1. Математическое ядро (Класс JacobiSolver)

Центральным компонентом программы является класс JacobiSolver, инкапсулирующий в себе логику метода простых итераций. В нем реализованы механизмы настройки точности (eps) и ограничения вычислительной трудоемкости (max_iter). Внутренние методы класса обеспечивают:

Механизм частичного выбора ведущего элемента (applyPivoting): выполняет перестановку строк для максимизации диагональных коэффициентов, что критически важно для обеспечения условий сходимости метода Якоби.

Итерационный цикл: реализует последовательное вычисление вектора приближений. Алгоритм спроектирован с использованием двух векторов состояний (текущего и предыдущего), что исключает влияние обновленных компонент на расчеты внутри одной итерации, строго соблюдая каноническую схему Якоби.

Контроль сходимости и невязки: метод getResidual вычисляет норму вектора ошибки $\|Ax - b\|$, позволяя оценить реальную точность полученного решения.

2. Блок диагностического анализа (Метод analyzeMatrix и структура MatrixStats)

Данный компонент отвечает за реализацию теоремы Кронекера-Капелли и проверку корректности постановки задачи.

Вычисление ранга: с помощью прямого хода метода Гаусса вычисляется ранг матрицы. Это позволяет на этапе препроцессинга выявить вырожденные системы, для которых решение методом Якоби априори невозможно.

Оценка ресурсов: структура MatrixStats аккумулирует данные о ранге, флаг вырожденности и объем оперативной памяти, выделяемой под хранение коэффициентов системы. Это обеспечивает прозрачность работы программы при обработке матриц сверхбольших порядков.

3. Интерфейс пользователя и логика управления (main.cpp)

Данный модуль реализует интерактивный режим взаимодействия. Его ключевой особенностью является строгая дисциплина выполнения: программа не допускает запуск итерационного процесса, если предварительный анализ выявил вырожденность матрицы. Модуль обеспечивает ввод данных, визуализацию процесса анализа и форматированный вывод итогового вектора решений вместе с показателями точности.

4. Модуль автоматизированного тестирования и генерации (test.cpp)

Для верификации устойчивости алгоритма реализован отдельный компонент, содержащий специализированные сценарии:

Генераторы специальных структур: функции для автоматического формирования матриц Гильберта (тест на плохую обусловленность) и разреженных диагональных матриц большой размерности (тест на производительность).

Система автоматического протоколирования: для каждого теста фиксируется время выполнения в миллисекундах, объем затраченной памяти и статус сходимости. Это позволяет проводить сравнительный анализ эффективности метода на различных классах задач: от малых переопределенных систем до разреженных структур размером 1000×1000 .

5. Структуры данных и типизация

В программе использованы динамические структуры данных на базе стандартной библиотеки шаблонов (STL). Матрицы представлены в виде векторов векторов (`vector<vector<double>>`), что обеспечивает гибкость при работе с неквадратными или разреженными системами, сохраняя при этом высокую скорость доступа к элементам в памяти. Все вычисления производятся

с двойной точностью (double), что необходимо для минимизации накопления погрешностей округления в итерационных методах.

Тестирование.

Код тестов приведен в Приложении А. Тестирование приведено в таблице 1 в Приложении Б.

Вывод.

В ходе работы был реализован и протестирован метод Якоби (метод простых итераций) для решения СЛАУ с поддержкой выбора ведущего элемента (частичный выбор по столбцу). Программа корректно определяет тип решения (единственное, бесконечно много, решений нет), выполняет проверку совместности системы и контроль невырожденности матрицы в процессе вычислений через ранговый анализ, проверку специальных строк и контроль диагональных элементов на итерациях. Проведённое тестирование на базовых примерах, матрице Гильберта и больших размерностях (включая 1000x1000) подтвердило корректность реализации, ожидаемую численную устойчивость на подходящих классах матриц и работоспособность на больших входных данных. Дополнительно получены практические характеристики времени и памяти, что позволило оценить реальную вычислительную стоимость метода для крупных разреженных систем.

ПРИЛОЖЕНИЕ А

Название файла: main.cpp

```
#include "jacobi_core.h"

int main() {
    // Устанавливаем локаль для корректного отображения символов
    (необязательно)
    setlocale(LC_ALL, "Russian");

    cout << "=== СВОБОДНЫЙ РЕЖИМ ВВОДА СЛУ ===" << endl;

    int n;
    cout << "Введите размерность квадратной матрицы (n): ";
    if (!(cin >> n) || n <= 0) {
        cout << "Ошибка: Некорректный размер." << endl;
        return 0;
    }

    Matrix A(n, Vector(n));
    Vector b(n);

    cout << "Введите элементы матрицы A (" << n << "x" << n << ")
    построчно:" << endl;
    for(int i = 0; i < n; i++) {
        for(int j = 0; j < n; j++) {
            cin >> A[i][j];
        }
    }

    cout << "Введите вектор b (" << n << " значений):" << endl;
    for(int i = 0; i < n; i++) cin >> b[i];

    JacobiSolver solver;

    // --- ЭТАП 1: ПРОВЕРКА НА ВШИВОСТЬ (ВЫРОЖДЕННОСТЬ) ---
    cout << "\n[!] Анализ системы на наличие единственного решения..." <<
endl;
    MatrixStats stats = solver.analyzeMatrix(A);

    /*
    ВНИМАНИЕ: Если stats.is_degenerate == true, значит Det = 0.
    В этом случае по вашему требованию мы ОТМЕНЯЕМ любые дальнейшие
    действия.
    */
    if (stats.is_degenerate) {
        cout << "-----"
endl;
        cout << "ОТМЕНА ОПЕРАЦИИ: Матрица системы является ВЫРОЖДЕННОЙ." <<
endl;
        cout << "Причина: Ранг матрицы (" << stats.rank << ") меньше
    размерности (" << n << ")." << endl;
        cout << "Математически невозможно найти единственное решение." <<
endl;
        cout << "-----"
endl;
        return 0; // Полный выход из программы
    }

    // --- ЭТАП 2: ЕСЛИ МАТРИЦА НЕВЫРОЖДЕННАЯ, ИДЕМ ДАЛЬШЕ ---
```

```

        cout << "-> Проверка пройдена: Матрица невырожденная (Ранг " <<
stats.rank << ")." << endl;
        cout << "-> Выделено памяти: " << stats.memory_bytes << " байт." <<
endl;
        cout << "-> Запуск итерационного процесса Якоби...\n" << endl;

        bool success = false;
        Vector x = solver.solve(A, b, success);

        if (success) {
            cout << "=== РЕЗУЛЬТАТ СХОДИМОСТИ ===" << endl;
            for(int i = 0; i < x.size(); i++) {
                cout << "x[" << i << "] = " << fixed << setprecision(6) << x[i]
<< endl;
            }
            cout << "Итоговая невязка (ошибка): " << solver.getResidual(A, x,
b) << endl;
        } else {
            cout << "=== ОШИБКА СХОДИМОСТИ ===" << endl;
            cout << "Матрица невырожденная, но метод простых итераций не
сошелся." << endl;
            cout << "Скорее всего, не выполнено условие диагонального
преобладания." << endl;
            cout << "Рекомендуется использовать метод Гаусса." << endl;
        }

        return 0;
    }
}

```

Название файла: test.cpp

```

#include "jacobi_core.h"

void run_test_case(string name, Matrix A, Vector b) {
    JacobiSolver solver;
    auto start = chrono::high_resolution_clock::now();

    // Анализ матрицы (ранг, невырожденность, память)
    MatrixStats stats = solver.analyzeMatrix(A);
    bool ok = false;

    // Решение методом Якоби
    Vector x = solver.solve(A, b, ok);

    auto end = chrono::high_resolution_clock::now();
    double time = chrono::duration<double, milli>(end - start).count();

    cout << left << setw(20) << name
        << " | Rank: " << setw(5) << stats.rank
        << " | Deg: " << (stats.is_degenerate ? "Yes" : "No ")
        << " | Res: " << setw(10) << (ok ?
to_string(solver.getResidual(A, x, b)) : "FAILED")
        << " | Time: " << setw(8) << time << "ms"
        << " | Mem: " << (stats.memory_bytes < 1024*1024 ?
to_string(stats.memory_bytes) + " B" :
to_string(stats.memory_bytes/1024/1024) + " MB")
        << endl;
}

```

```

}

int main() {
    cout << "=== ЗАПУСК АВТОМАТИЧЕСКИХ ТЕСТОВ ===" << endl;

    // Тест 5: Гильберт 5x5
    {
        int n = 5;
        Matrix A(n, Vector(n)); Vector b(n, 1.0);
        for(int i=0; i<n; i++) for(int j=0; j<n; j++) A[i][j] =
1.0/(i+j+1);
        run_test_case("Hilbert 5x5", A, b);
    }

    // Тест 6: Гильберт 8x8
    {
        int n = 8;
        Matrix A(n, Vector(n)); Vector b(n, 1.0);
        for(int i=0; i<n; i++) for(int j=0; j<n; j++) A[i][j] =
1.0/(i+j+1);
        run_test_case("Hilbert 8x8", A, b);
    }

    // Тест 7: Диагональная 1000x1000
    {
        int n = 1000;
        Matrix A(n, Vector(n, 0)); Vector b(n);
        for(int i=0; i<n; i++) {
            A[i][i] = i+1;
            b[i] = pow(i+1, 2);
        }
        run_test_case("Diag 1000x1000", A, b);
    }

    // Тест 8: Нулевая матрица
    {
        run_test_case("Zero 2x2", {{0,0},{0,0}}, {1,0});
    }

    // Тест 9: Переопределенная 3x2 (совместная)
    {
        run_test_case("Overdet 3x2", {{1,2},{2,1},{3,0}}, {5,4,3});
    }

    // Тест 10: Диагональная 10000x10000
    {
        int n = 10000;
        // Внимание: выделение памяти под 10000x10000 типа double займет
~800 МБ RAM
        Matrix A(n, Vector(n, 0));
        Vector b(n);
        for(int i=0; i<n; i++) {
            A[i][i] = i+1;
            b[i] = pow(i+1, 2);
        }
        run_test_case("Diag 10000x10000", A, b);
    }
}

```

```

    }
    return 0;
}

```

Название файла: jacobi_core.h

```

#ifndef JACOBI_CORE_H
#define JACOBI_CORE_H

#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>
#include <chrono>
#include <algorithm>

using namespace std;

typedef vector<vector<double>> Matrix;
typedef vector<double> Vector;

// Структура для хранения результатов анализа матрицы
struct MatrixStats {
    int rank;
    bool is_degenerate;
    double memory_bytes;
};

class JacobiSolver {
public:
    double eps;
    int max_iter;

    JacobiSolver(double e = 1e-7, int iter = 10000) : eps(e),
max_iter(iter) {}

    // 1. Метод вычисления ранга и проверки на вырожденность (методом
Гаусса)
    MatrixStats analyzeMatrix(Matrix A) {
        int n = A.size();
        int m = A[0].size();
        int rank = 0;
        vector<bool> row_used(n, false);

        for (int i = 0; i < m; ++i) {
            int j;
            for (j = 0; j < n; ++j) {

```

```

        if (!row_used[j] && abs(A[j][i]) > 1e-12) break;
    }

    if (j != n) {
        rank++;
        row_used[j] = true;
        for (int p = i + 1; p < m; ++p) A[j][p] /= A[j][i];
        for (int k = 0; k < n; ++k) {
            if (k != j && abs(A[k][i]) > 1e-12) {
                for (int p = i + 1; p < m; ++p)
                    A[k][p] -= A[j][p] * A[k][i];
            }
        }
    }
}

double mem = (double)(n * m * sizeof(double) + n * sizeof(double));
return {rank, (n == m ? rank < n : true), mem};
}

// 2. Метод выбора ведущего элемента (частичный выбор по столбцу)
// Переставляет строки, чтобы на диагонали были максимальные элементы
для сходимости
void applyPivoting(Matrix& A, Vector& b) {
    int n = A.size();
    for (int i = 0; i < n; ++i) {
        int max_row = i;
        for (int k = i + 1; k < n; ++k) {
            if (abs(A[k][i]) > abs(A[max_row][i])) max_row = k;
        }
        swap(A[i], A[max_row]);
        swap(b[i], b[max_row]);
    }
}

// 3. Основной алгоритм Якоби
Vector solve(Matrix A, Vector b, bool& success) {
    int rows = A.size();
    int cols = A[0].size();

    // Для неквадратных систем (Тест 9) берем размер по количеству
    неизвестных
    int n = cols;
    Vector x(n, 0.0);
    Vector x_new(n, 0.0);

    applyPivoting(A, b);

    for (int iter = 0; iter < max_iter; ++iter) {
        for (int i = 0; i < n; ++i) {

```

```

        // Проверка деления на ноль
        if (abs(A[i][i]) < 1e-18) {
            success = false;
            return x;
        }

        double sum = 0;
        for (int j = 0; j < n; ++j) {
            if (i != j) sum += A[i][j] * x[j];
        }
        x_new[i] = (b[i] - sum) / A[i][i];
    }

    // Проверка сходимости по норме разности
    double diff = 0;
    for (int i = 0; i < n; ++i) diff += pow(x_new[i] - x[i], 2);

    if (sqrt(diff) < eps) {
        success = true;
        return x_new;
    }
    x = x_new;

    // Проверка на расходимость
    if (diff > 1e20) { success = false; return x; }
}

success = false;
return x;
}

// Расчет невязки ||Ax - b||
double getResidual(const Matrix& A, const Vector& x, const Vector& b) {
    double max_err = 0;
    for (int i = 0; i < A.size(); ++i) {
        double s = 0;
        for (int j = 0; j < x.size(); ++j) s += A[i][j] * x[j];
        max_err = max(max_err, abs(s - b[i]));
    }
    return max_err;
}
};

#endif

```

ПРИЛОЖЕНИЕ В

ТЕСТИРОВАНИЕ

Результаты тестирования

№	Ввод (с описанием)	Вывод (включая решение)	Время	Память
5	Матрица Гильберта 5×5 : $H[i][j] = 1/(i+j+1)$, $x_{\text{exact}} = [1,1,1,1,1]$ ($\text{cond} \approx 5 \cdot 10^5$)	Единственное решение, $\text{rank} = 5$, невырожденность: да, тест провален, метод разошёлся из-за плохой обусловленности	0.049 мс	240 Б
6	Матрица Гильберта 8×8 : $H[i][j] = 1/(i+j+1)$, $x_{\text{exact}} = [1, \dots, 1]$ ($\text{cond} \approx 1.5 \cdot 10^{10}$)	Единственное решение, $\text{rank} = 8$, невырожденность: да, тест провален, условие сходимости не выполнено	0.037 мс	576 Б
7	Диагональная 1000×1000 : $A[i][i] = i+1$, $b[i] = (i+1)^2$, $x_{\text{exact}}[i] = i+1$ (проверка масштаба)	Единственное решение, $\text{rank} = 1000$, невырожденность: да, погрешность $< 1e-6$	107.9 мс	7.64 МБ
8	Нулевая матрица, $b \neq 0$: $A = [[0,0],[0,0]]$, $b = [1,0]$ (крайний случай)	Решений нет, $\text{rank} = 0$, невырожденность: нет, x отсутствует	0.006 мс	48 Б
9	Переопределённая 3×2 : 3 уравнения, 2 неизвестных, все совместны, $x_{\text{exact}} = [1,2]$	Единственное решение, $\text{rank} = 2$, невырожденность: нет (не квадратная), $x =$	0.009 мс	72 Б

		[1.000000, 2.000000]		
10	Диагональная 10000×10000 : $A[i][i] = i+1$, $b[i] = (i+1)^2$, $x_{\text{exact}}[i] = i+1$ (проверка масштаба)	Единственное решение, $\text{rank} = 1000$, невырожденность: да, погрешность $< 1e-6$	10.3 с	763.02 МБ

Таблица 1 – Тестирование алгоритма.